

¿Qué es una relación de Asociación en UML? Desarrollar la explicación con un ejemplo.

Una **relación de asociación** en **UML (Unified Modeling Language)** es una relación estructural que indica que **dos o más clases están conectadas porque sus objetos necesitan interactuar o comunicarse entre sí**. Esta relación representa que una clase conoce o utiliza a otra para cumplir alguna función dentro del sistema, pero **cada clase puede existir de manera independiente**, es decir, ninguna depende de la otra para ser creada o destruida.

La asociación es la relación más común en un **Diagrama de Clases** y se representa mediante una **línea continua** que une las clases. Sobre esa línea pueden indicarse las **cardinalidades (multiplicidades)**, que especifican cuántos objetos de una clase pueden relacionarse con objetos de la otra.

EJEMPLO

Supongamos un sistema para gestionar una empresa con sus empleados.

Existen las clases:

- **Sucursal**
- **Empleado**

La relación es que **una sucursal tiene empleados** y **cada empleado trabaja en una sucursal**.

Se representa de la siguiente manera:

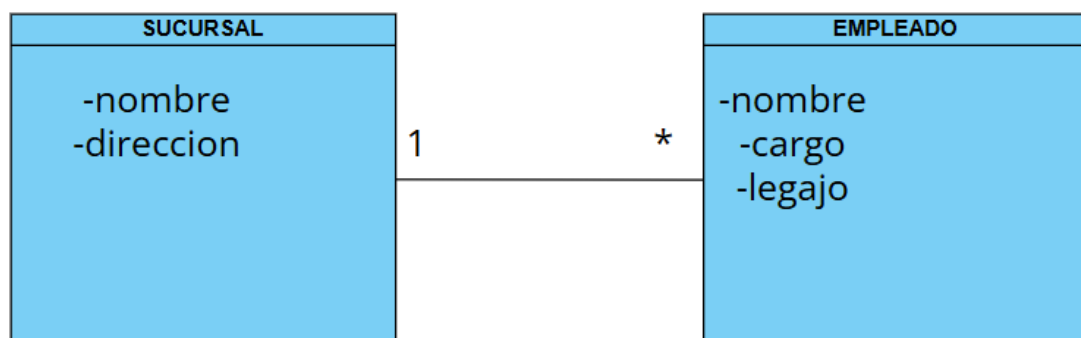


DIAGRAMA HECHO EN: <https://online.visual-paradigm.com/app/diagrams/>

En código, esta asociación podría reflejarse así:

```
class Sucursal {  
    private String nombre;
```

```
private String direccion;  
private List<Empleado> empleados;  
}
```

```
class Empleado {  
    private String nombre;  
    private int legajo;  
    private String cargo;  
    private Sucursal sucursal;  
}
```

¿Qué es una relación de Agregación en UML? Desarrollar la explicación con un ejemplo.

Una **relación de agregación** en **UML (Unified Modeling Language)** es un tipo especial de **asociación** que representa una relación **"todo–parte"**, donde un objeto (el **todo**) está formado por uno o varios objetos (las **partes**). Sin embargo, **las partes pueden existir de manera independiente del todo**. Es decir, si el objeto principal deja de existir, los objetos que lo componían pueden seguir existiendo y ser utilizados por otros objetos.

En un **Diagrama de Clases**, la agregación se representa mediante una **línea continua con un rombo blanco (◊)** en el extremo de la clase que representa el **todo**.

EJEMPLO

Supongamos un sistema para gestionar una empresa con sus sucursales.

Las clases son:

- **Empresa**
- **Sucursal**

Una empresa está formada por varias sucursales, pero **una sucursal puede seguir existiendo aunque la empresa deje de utilizarla**, por ejemplo porque fue vendida a otra empresa o pasó a formar parte de otra organización.

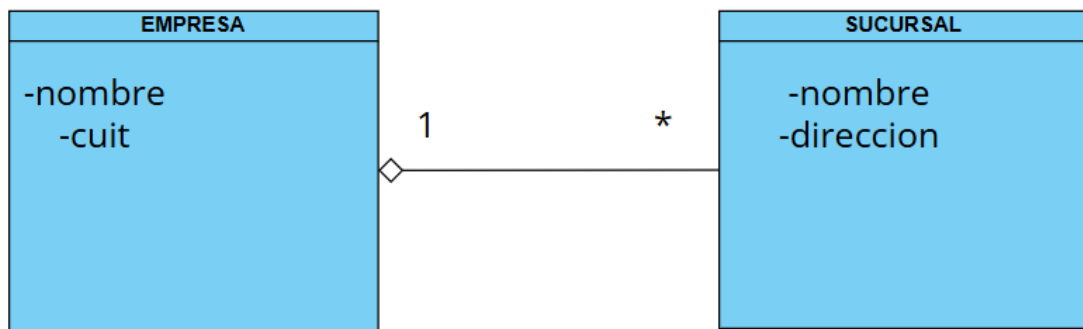


DIAGRAMA HECHO EN: <https://online.visual-paradigm.com/app/diagrams/>

En código, esta agregación podría implementarse de la siguiente manera:

```
class Empresa {
    private String nombre;
    private String cuit;
    private List<Sucursal> sucursales;
}
```

```
class Sucursal {
    private String nombre;
    private String direccion;
}
```

¿Qué es una relación de Composición en UML? Desarrollar la explicación con un ejemplo.

Una **relación de composición** en **UML (Unified Modeling Language)** es un tipo especial de **agregación** que representa una relación **"todo-parte" muy fuerte**, donde las partes **no pueden existir de forma independiente del todo**. Esto significa que el objeto principal es el responsable del ciclo de vida de los objetos que lo componen: si el objeto principal se crea, normalmente crea también sus partes; y si se elimina, **las partes también se eliminan**.

En un **Diagrama de Clases**, la composición se representa mediante una **línea continua con un rombo negro (◆)** en el extremo de la clase que representa el **todo**.

EJEMPLO

Supongamos un sistema para gestionar una empresa con sus sucursales y oficinas.

Las clases son:

- **Sucursal**
- **Oficina**

Una sucursal está formada por varias oficinas. En este caso, las **oficinas no tienen sentido sin la sucursal a la que pertenecen**. Si una sucursal deja de existir, también desaparecen las oficinas que forman parte de ella.

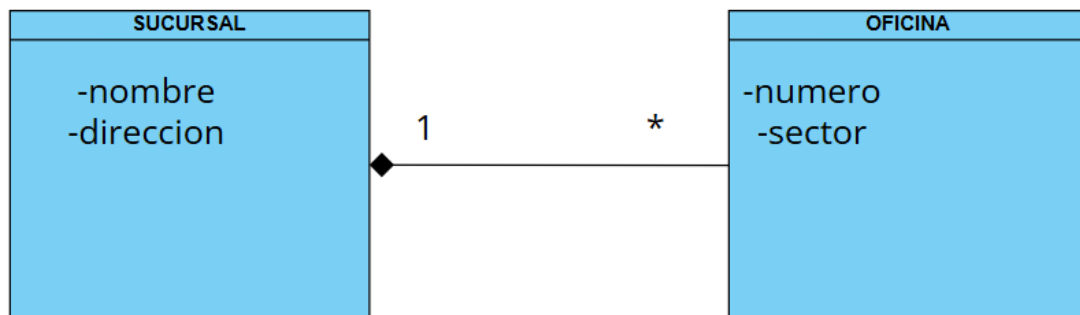


DIAGRAMA HECHO EN: <https://online.visual-paradigm.com/app/diagrams/>

En código, esta agregación podría implementarse de la siguiente manera:

```
class Sucursal {
    private String nombre;
    private String direccion;
    private List<Oficina> oficinas;
}
```

```
class Oficina {
    private int numero;
    private String sector;
}
```

¿Qué es una relación de Herencia en UML? Desarrollar la explicación con un ejemplo.

Una **relación de herencia** en **UML (Unified Modeling Language)**, también llamada **generalización**, es una relación que permite que una clase (**subclase** o **clase hija**) herede

los atributos y métodos de otra clase (**superclase** o **clase padre**). De esta forma, las clases hijas reutilizan las características comunes de la clase padre y, además, pueden incorporar nuevos atributos o métodos, o redefinir los existentes.

La herencia se utiliza cuando existe una relación de tipo "**es un(a)**" (*is-a*). Por ejemplo, un **Gerente es un Empleado**, por lo que comparte todas las características de un empleado, pero también posee características propias.

En un **Diagrama de Clases**, la herencia se representa mediante una **línea continua con una flecha de triángulo blanco** apuntando hacia la **superclase**.

EJEMPLO

Supongamos un sistema para administrar los empleados de una empresa.

Existe una clase general **Empleado**, de la cual heredan las clases **Gerente** y **Vendedor**.

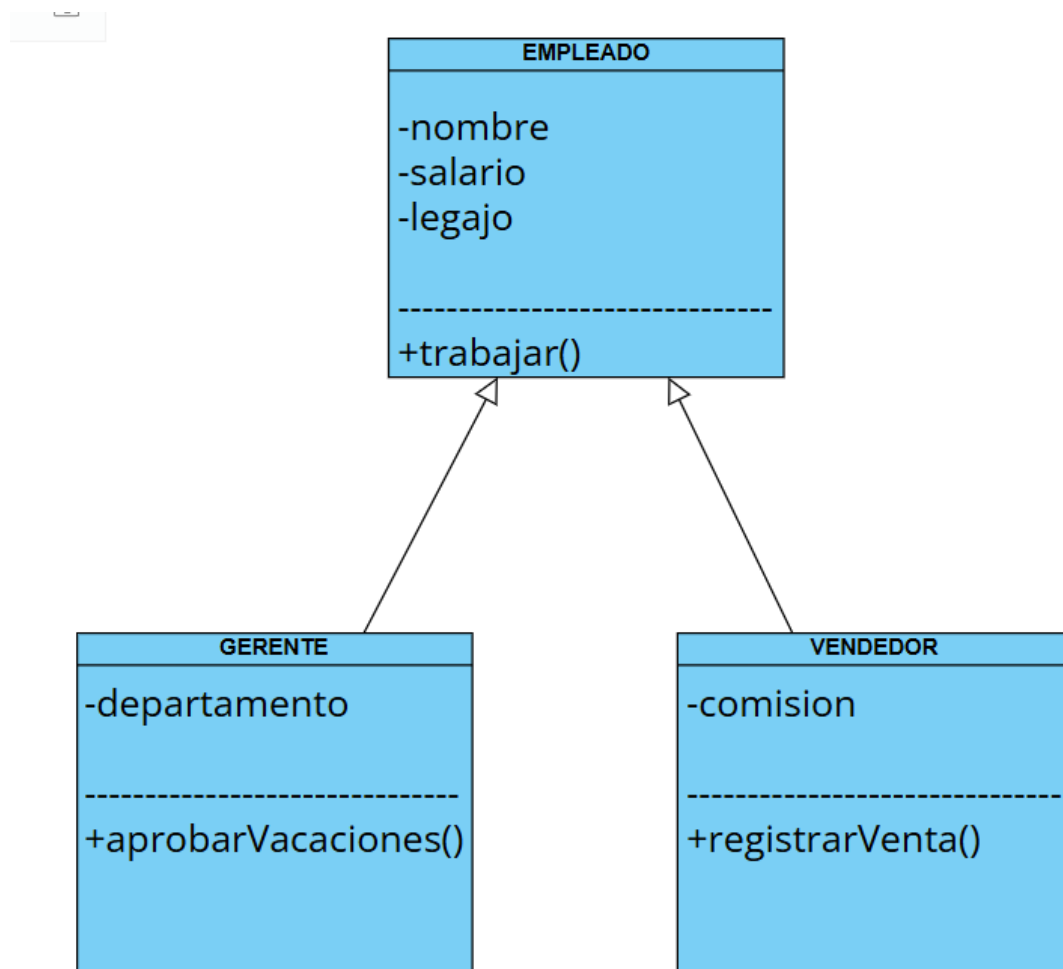


DIAGRAMA HECHO EN: <https://online.visual-paradigm.com/app/diagrams/>

En código Java, esta herencia podría implementarse así:

```
class Empleado {  
    private int legajo;  
    private String nombre;  
    private double salario;  
  
    public void trabajar() {  
        System.out.println("El empleado está trabajando.");  
    }  
}
```

```
class Gerente extends Empleado {  
    private String departamento;  
  
    public void aprobarVacaciones() {  
        System.out.println("Vacaciones aprobadas.");  
    }  
}
```

```
class Vendedor extends Empleado {  
    private double comision;  
  
    public void registrarVenta() {  
        System.out.println("Venta registrada.");  
    }  
}
```